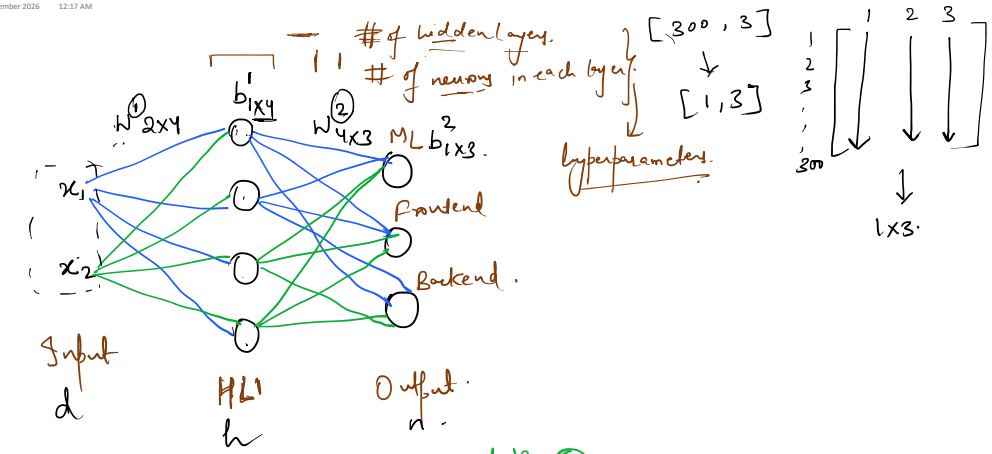
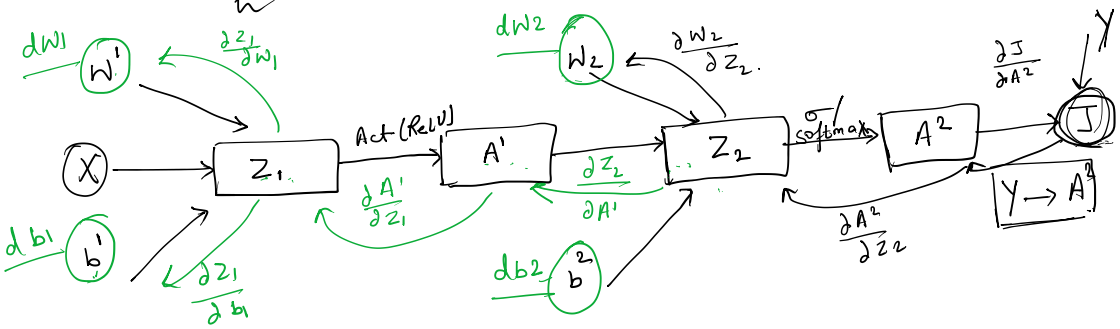
Training Multi-Layer Networks

Tuesday, 29 September 2026 12:17 AM





$$W^1_{2\times4} = \begin{matrix}① \\ ②\end{matrix}\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \end{bmatrix}$$

$$b^1_{1\times4} = \begin{bmatrix} b_1 & b_2 & b_3 & b_4 \end{bmatrix}$$

$\log(0.9 \times 0.33)$

$\log 0.9 + \log 0.33.$

1 2 3 ... K.

Why is $\frac{dJ}{dW} = (P - Y)X$? (Derivation of $P - Y$)

1 Setup: forward flow (one example)

$\boldsymbol{Z}$ (logits) → Softmax: $\boldsymbol{P} = \text{softmax}(Z)$ (probabilities) → Cross-Entropy Loss: $J = -\sum_i Y_i \log P_i$

We want to compute the gradient of the loss w.r.t. the logits Z: $\frac{\partial J}{\partial Z}$. This will give us $P - Y$.

2 Softmax and Cross-Entropy

Softmax: $P_i = \frac{e^{Z_i}}{\sum_j e^{Z_j}}$

P_i is the probability of class i.

Cross-Entropy Loss (for one example): $J = -\sum_i Y_i \log P_i$

Y is one-hot encoded (only the correct class is 1, rest are 0).

3 Step 1: Substitute softmax into the loss

$$J = -\sum_i Y_i \log\left(\frac{e^{Z_i}}{\sum_j e^{Z_j}}\right)$$ Replace P_i with softmax formula

$$= -\sum_i Y_i \left[\log(e^{Z_i}) - \log\left(\sum_j e^{Z_j}\right)\right]$$

$$= -\sum_i Y_i \left[Z_i - \log\left(\sum_j e^{Z_j}\right)\right]$$ Using $\log\frac{a}{b} = \log a - \log b$

$$= -\sum_i Y_i Z_i + \left(\sum_i Y_i\right)\log\left(\sum_j e^{Z_j}\right)$$ Take log terms out of the sum

Since Y is one-hot, $\sum_i Y_i = 1$, so $J = -\sum_i Y_i Z_i + \log\left(\sum_j e^{Z_j}\right)$ (Y_k, P_k.)

4 Step 2: Differentiate w.r.t. Z_k

a Derivative of the first term: $\frac{\partial}{\partial Z_k}\left(-\sum_i Y_i Z_i\right) = -Y_k$

Only the kth term survives. Y_k is 1 for the correct class, else 0.

b Derivative of the second term: $\frac{\partial}{\partial Z_k}\log\left(\sum_j e^{Z_j}\right) = \frac{1}{\sum_j e^{Z_j}} \cdot e^{Z_k} = \frac{e^{Z_k}}{\sum_j e^{Z_j}} = P_k$

This is exactly the softmax probability P_k.

5 Step 3: Combine the two parts

$$\frac{\partial J}{\partial Z_k} = -Y_k + P_k$$

$$\frac{\partial J}{\partial Z_k} = P_k - Y_k$$

For all classes (vector form): $\frac{\partial J}{\partial Z} = P - Y$

- Shape: same as Z $(m \times K)$
- Meaning:
  - Positive for incorrect classes (push down logits)
  - Negative for the correct class (push up logits)

6 Example (intuition)

Class	Prediction P	Actual Y	$P - Y$	Effect on logit
1	0.2	0	0.2	Push down ↓
2	0.5	1	−0.5	Push up ↑
3	0.3	0	0.3	Push down ↓

Intuition:
- For incorrect classes: $P - Y > 0$ → decrease their logits
- For the correct class: $P - Y < 0$ → increase its logit

7 Gradient w.r.t. weights

Forward relation: $Z = XW$

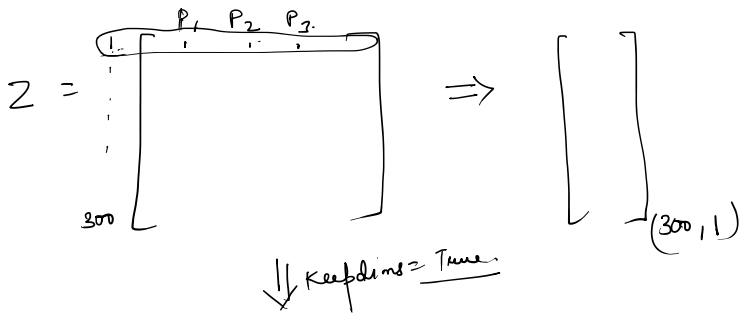
Using chain rule: $\frac{\partial J}{\partial W} = X^T \frac{\partial J}{\partial Z}$

Substitute $\frac{\partial J}{\partial Z} = P - Y$: $\frac{\partial J}{\partial W} = X^T(P - Y)$

For m examples (batch): $\frac{\partial J}{\partial W} = \frac{X^T(P - Y)}{m}$

Shapes (batch of m examples)
- X : $(m, n_{features})$
- W : $(n_{features}, n_{classes})$
- $P - Y$: $(m, n_{classes})$
- dW : $(n_{features}, n_{classes})$

Final layer.
Multi class classification → Softmax.
Binary → Sigmoid.



⇓ Keepdims = True.

$Z^2 = A^1 * W^2 + b^2$.

$$\frac{\partial J}{\partial W^2} = \underbrace{\frac{\partial J}{\partial A^2} \times \frac{\partial A^2}{\partial Z^2}}_{dZ^2} \times \frac{\partial Z^2}{\partial W^2}$$

A^1

$(A^2 - Y)$

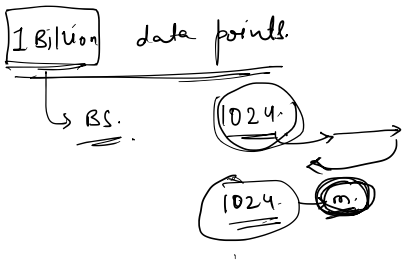
dZ^2.

Probs → [0.33 | 0.33 + 1 | 0.33]

Y(1) → [0 | | 0]

0.33 | -0.67 | 0.33.

$$W^{new} = W_{old} - \eta \left[\frac{\partial J}{\partial W}\right]$$



$Z_1 = X \cdot W^1 + b^1$

$$\frac{\partial Z_1}{\partial W^1} = X \qquad \frac{\partial Z_1}{\partial b_1} = 1$$

$ReLU(Z^1) = \max(0, Z_1)$.

$$\begin{cases} A^1 = Z^1 & \text{if } Z^1 > 0 \\ A^1 = 0 & \text{if } Z^1 \le 0. \end{cases}$$

$dA^1 = [1, 0]$.

$\frac{\partial Z_1}{\partial}$

.backward()

$$W^{new} = W^{old} - \eta \frac{\partial J}{\partial W}$$

$$b^{new} = b^{old} - \eta \frac{\partial J}{\partial b}$$

↓

Lr.